INFORME TÉCNICO DE BASE DE DATOS

Migración, Evaluación y Análisis Comparativo: PostgreSQL vs. MongoDB

Sistema: Food Store E-commerce

Entorno Relacional: PostgreSQL (DBeaver)

Archivos Entregados: `consultas.sql`, `esquema_parte1.json`, `migracion_y_consultas.js`, `analisis_pg_vs_mongo.md`

Muestra de Migración: Pedidos IDs 10108 a 10112

Entorno NoSQL: MongoDB 6.0+ (Compass / _mongosh)

PARTE 1: INTEGRIDAD Y ESTRUCTURA DE MIGRACIÓN

Para la migración del subconjunto de prueba desde el modelo relacional hacia la base documental en MongoDB, se seleccionó una muestra representativa de 5 pedidos (IDs 10108 al 10112) que abarca diversidad de estados (`CONFIRMADO`, `CANCELADO` y `PENDIENTE`) y distintas formas de pago.

Consigna 3: Garantía de Integridad y Reglas de Negocio en MongoDB

- **Mapeo de Clave Primaria:** Se conservó el identificador secuencial original mapeándolo directamente al atributo obligatorio `_id` de MongoDB (ej. `_id: 10108`), evitando el uso de identificadores autogenerados de tipo `ObjectId` para mantener la correspondencia exacta entre ambas bases de datos.
- **Integridad de Dominio (Estados):** La restricción relacional `ENUM` (`estado_pedido`) se representó como cadenas de texto (`String`) estandarizadas en mayúsculas. Ante documentos sin este atributo, la regla de dominio asigna programáticamente `"PENDIENTE"` como valor predeterminado.
- **Modelado Embebido:** Los renglones de la tabla relacional `detalle_pedido` se estructuraron dentro del arreglo `detalle_pedido: [...]` en la propia orden, eliminando la dispersión de datos en múltiples tablas.

PARTE 2: EJECUCIÓN Y COMPARACIÓN DE CONSULTAS

Se ejecutaron consultas equivalentes en ambos motores sobre la muestra migrada para contrastar la estructura del conjunto de resultados obtenido en PostgreSQL frente al comportamiento documental de MongoDB.

Consulta / Filtro Evaluado	Cantidad en PostgreSQL	Cantidad en MongoDB	Coincidencia de IDs
a) Estado seleccionado (`PENDIENTE`)	15 filas	3 documentos	Sí (10110, 10111, 10112)
b) Producto seleccionado (`id_producto: 10`)	1 fila	1 documento	Sí (10112)
c) Monto total superior (`total > 20000`)	5 filas	1 documento	Sí (10112)

Consultas Ejecutadas en el Laboratorio:

1. Filtrar por Estado (`PENDIENTE`):

```
-- PostgreSQL (CTE desnormalizada) WITH pedido_detalle AS ( SELECT p.id_pedido, p.fecha, p.es
tado, p.forma_pago, p.total, p.id_usuario, d.id_detalle, d.cantidad, d.precio_unitario, d.sub
total, d.id_producto FROM pedido p JOIN detalle_pedido d ON p.id_pedido = d.id_pedido WHERE
p.id_pedido IN (10108, 10109, 10110, 10111, 10112) ORDER BY p.id_pedido ) SELECT * FROM pedid
o_detalle pd WHERE pd.estado = 'PENDIENTE'; // MongoDB (_mongosh) db.pedidos.find({ "estado":
"PENDIENTE" });
```

2. Buscar por Producto Específico (`id_producto = 10`):

```
-- PostgreSQL (CTE desnormalizada) WITH pedido_detalle AS ( ... ) SELECT * FROM pedido_detall
e pd WHERE pd.id_producto = 10; // MongoDB (_mongosh) db.pedidos.find({ "detalle_pedido.produ
cto.id_producto": 10 });
```

3. Filtrar por Monto Total (`total > 20000`):

```
-- PostgreSQL (CTE desnormalizada) WITH pedido_detalle AS ( ... ) SELECT * FROM pedido_detall
e pd WHERE pd.total > 20000; // MongoDB (_mongosh) db.pedidos.find({ "total": { $gt: 20000 }
});
```

PARTE 3: EVALUACIÓN DE TECNOLOGÍA E INTELIGENCIA ARTIFICIAL

Consigna 6: Prompt Enviado al Agente de IA

> *"Actuá como un arquitecto de bases de datos senior. Analizá el sistema de la tienda Food Store para comparar PostgreSQL y MongoDB en cuanto al almacenamiento y la consulta de pedidos. Tené en cuenta el modelo actual (donde los pedidos se dividen en cabecera pedido y renglones en detalle_pedido), la necesidad del negocio de realizar accesos frecuentes a pedidos completos (cabecera + detalle de productos) y el requerimiento crítico de conservar la coherencia de cantidades, precios unitarios y subtotales acumulados en cada transacción. Explicá las ventajas, desventajas y el impacto técnico de cada motor para este escenario."*

Consigna 7: Contrastación con Laboratorio y Conclusión Técnica

1. Contrastación de Afirmaciones de la IA:

- **Afirmación 1 de la IA:** *"En PostgreSQL, recuperar un pedido completo obliga a realizar JOINs entre la cabecera y el detalle, duplicando la información de la cabecera por cada renglón de detalle en la grilla devuelta."*
  Contrastación con el laboratorio: VERIFICADO. Al ejecutar la consulta SQL sobre la CTE `pedido_detalle` para los 5 pedidos de muestra en DBeaver, PostgreSQL devolvió **25 filas** planas, duplicando los atributos de la orden por cada ítem. En cambio, en MongoDB la consulta devolvió únicamente **5 documentos BSON unificados**, confirmando la eliminación de redundancia en el transporte de datos.

- **Afirmación 2 de la IA:** *"MongoDB permite atomicidad a nivel de un solo documento embebido, pero carece de restricciones declarativas nativas como CHECK (subtotal = cantidad * precio_unitario) o FOREIGN KEY, delegando la integridad matemática a la capa de aplicación."*
  Contrastación con documentación oficial y laboratorio: VERIFICADO. Según la documentación de MongoDB (*Schema Validation*), aunque `$jsonSchema` permite validar tipos de datos, no expresa reglas aritméticas cruzadas entre atributos de un arreglo ni integridad referencial. En PostgreSQL, las reglas CHECK y `NUMERIC(10,2)` garantizan la coherencia financiera directamente dentro del motor.

2. Conclusión Técnica (Estrategia de Almacenamiento Recomendada):

> Para el sistema *Food Store*, la estrategia óptima es adoptar una **arquitectura híbrida (persistencia políglota)**. Se recomienda **mantener PostgreSQL como la base de datos OLTP primaria**, ya que su motor relacional garantiza la *coherencia financiera e integridad referencial nativa* mediante restricciones `CHECK (subtotal = cantidad * precio_unitario)`, tipos `NUMERIC` exactos y transacciones ACID multi-tabla, previniendo inconsistencias durante el checkout.
>
> En paralelo, se propone **utilizar MongoDB como almacén de lectura/documental para el historial de pedidos confirmados**. Dado que el acceso a pedidos completos es de alta frecuencia, consultar documentos embebidos en MongoDB ofrece *mayor eficiencia de lectura al eliminar los JOINs relacionales* y reducir el volumen de datos redundantes transferidos por la red. De este modo, se combina la rigurosidad operacional de PostgreSQL con la velocidad de respuesta de MongoDB.